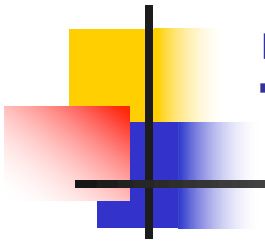


# 言語処理系 第2回
# 字句解析

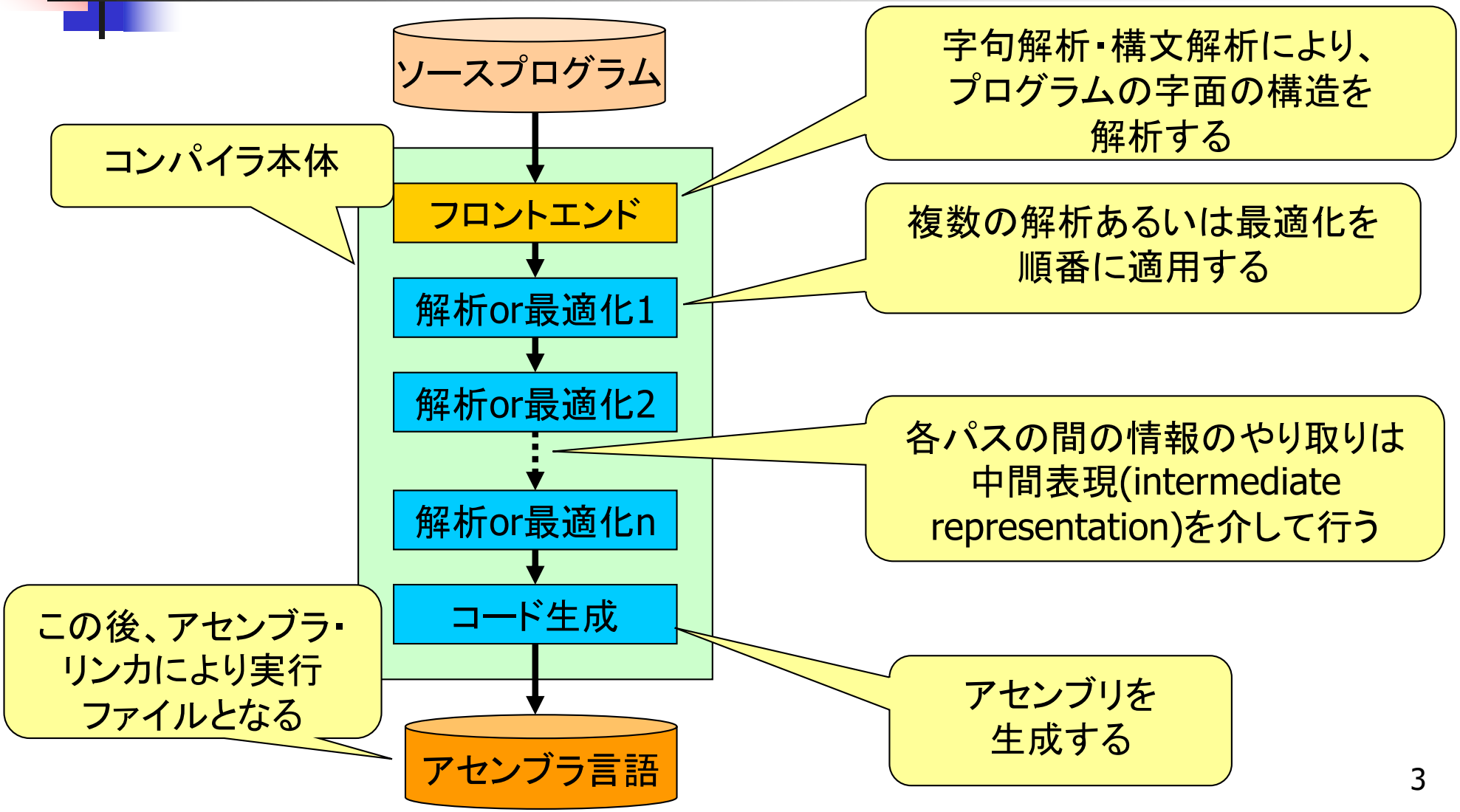
木村啓二
keiji@waseda.jp

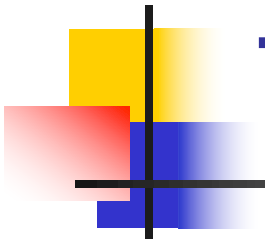


# 字句解析・構文解析

- コンパイラのフロントエンド部の処理
- ソースプログラムはファイル上の単なる文字列
  - 文字列から「書いてあること」を解析する必要がある
- 字句解析（今回の分）
  - ソースプログラムから単語を切り出す
- 構文解析（次回と次々回の分）
  - 式の解析が大きな動機の一つ
  - 字句解析で切り出した単語のつながりから、構文規則(syntax)をチェックする
    - 字句間のつながり方（式の場合、演算順序も表現する）をチェックする
  - さらにプログラムを内部表現に落とし込む

# コンパイラの基本構成


ソースプログラム
コンパイラ本体
フロントエンド
解析or最適化1
解析or最適化2
解析or最適化n
コード生成
アセンブラ言語
字句解析・構文解析により、プログラムの字面の構造を解析する
複数の解析あるいは最適化を順番に適用する
各パスの間の情報のやり取りは中間表現(intermediate representation)を介して行う
アセンブリを生成する
この後、アセンブラ・リンカにより実行ファイルとなる



# フロントエンド

- ソースプログラムを読み取る部分
- 字句解析 (lexical analyser, lexer, scanner)
  - ソースプログラムから単語(token)を切り出す
- 構文解析 (parser)
  - 字句解析で切り出した単語のつながりから、構文規則(syntax)をチェックする
  - ついでにプログラムを内部表現に落とし込む
- 文法規則から字句解析器・構文解析器を自動生成するツールがあります
  - lex (flex), yacc (bison), ANTLR

# 字句解析のイメージ
# プログラムから単語を切り出す

```
int
foo ( int a )
{
    int val ;

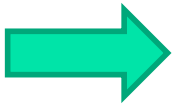
    if ( a > 0 ) {
        val = 10 * ( a + 3 ) ;
    } else {
        val = 10 * a + 3 ;
    }

    return  val ;
}
```

# もう少し字句解析のイメージ
# 単語を切り出しその種類も特定する

```
int val ;

if ( a > 0 )
    val = 10 * ( a + 3 ) ;
else
…
```



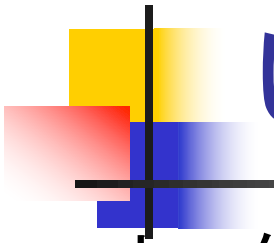
```
int: 予約語"int"
val: 識別子(val)
;: 記号"セミコロン"
if: 予約語"if"
(: 記号"開き丸括弧"
a: 識別子(a)
>: 記号"大なり"
0: 定数(0)
): 記号"閉じ丸括弧"
val: 識別子(val)
=: 記号"等号"
10: 定数(10)
*: 記号"アステリスク"
(: 記号"開き丸括弧"
a: 識別子(a)
+: 記号"加算"
3: 定数(3)
): 記号"閉じ丸括弧"
;: 記号"セミコロン"
else: 予約語"else"
…
```

- 予約語はプログラム言語が構文で予め使うことを指定している単語
- 識別子はユーザが定義できる単語
  変数や関数名など
- 記号や予約語は各々別々の記号や予約語として抽出される
- 識別子や定数は実際の名前や定数値と組み合わせて管理される

# 字句解析に必要な処理

```
int: 予約語"int"
val: 識別子(val)
;: 記号"セミコロン"
if: 予約語"if"
(: 記号"開き丸括弧"
a: 識別子(a)
>: 記号"大なり"
0: 定数(0)
): 記号"閉じ丸括弧"
val: 識別子(val)
=: 記号"等号"
10: 定数(10)
*: 記号"アステリスク"
(: 記号"開き丸括弧"
a: 識別子(a)
+: 記号"加算"
3: 定数(3)
): 記号"閉じ丸括弧"
;: 記号"セミコロン"
else: 予約語"else"
…
```

- 予約語と記号は予め何が存在するか決まっている
  - 読み取った文字列が**予め決められた単語と同一か**どうかチェックすれば良い
- 識別子や定数は「具体的に」どのような単語が出てくるかわからない
  - **ルールは決められる**
    - 整数の定数：0-9の数字の羅列
    - 識別子：'_'か英字で始まり'_'か英字か数字の羅列
  - 読み取った文字列が**予め決められた「ルール」に従っているか**どうかチェックすれば良い



# いくつかの用語

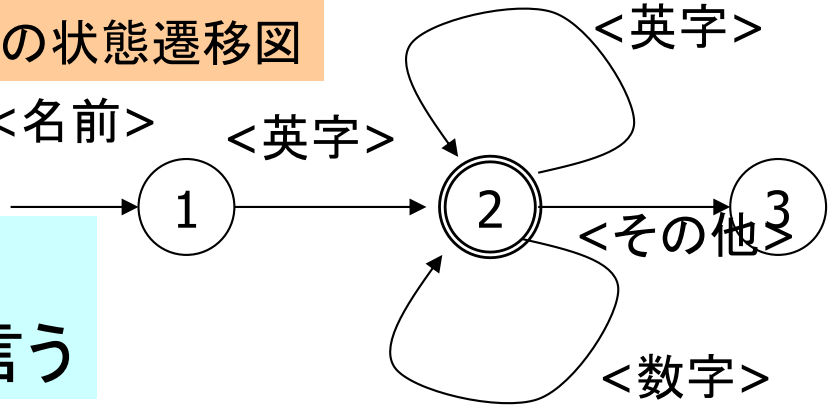
- トークン（token）
  - トークン名（識別子等）と属性（実際の識別子等）の組み合わせ
  - 例：<識別子, var>, <定数, 10>, <予約語"if">, <記号"等号">
    - 予約語や記号は属性無し
    - 識別子の属性はシンボルテーブル（後の回で扱う）の場合もある
- パターン
  - 各単語の種類を定義するルール
  - 予約語や記号はそれらそのもの
  - 整数の定数なら“0−9”の数字の羅列
  - 識別子なら"_か英字で始まり_か英字か数字の羅列"
- 語彙素（lexeme）
  - 字句解析器がプログラムから読み取った実際の単語
    - 「トークン」の例にあるvar, 10, ifなど

**字句解析器：ソースプログラムからパターンに従ってトークンを抽出する**

# 字句読み取り
# 基本的な方針

- 単語(字句)のルールをある表記法(後述)により定義する
- 読み込んだ文字列が定義に合致するかどうかチェックする
  - 定義から導出した状態遷移機械が受理すれば合致
- 例:
  <名前> → <英字>(<英字>|<数字>)*
  - 「名前」は「英字」で始まり0個以上の「英字」あるいは「数字」の繰り返し
  - waseda, version2202c1等

<名前>の状態遷移図




読み取った字句を
トークン(token)と言う

# <名前>の読み取りコード例(C)

```
index = 0;
nextState = 1;
while (1) {
  switch (nextState) {
  case 1:
    if (isalpha(ch)) {
      name[index++] = ch;
      nextState = 2;
    } else {
      error();
    }
    break;
  case 2:
    ch = fgetc(in);
    if (isalpha(ch) || isdigit(ch)) {
       name[index++] = ch;
       nextState = 2;
    } else {
       nextState = 3;
    }
    break;
  case 3:
    return name;
  }
}
```

最初の1文字はすでに
読み込まれている

読み込んだ文字が
アルファベットなら

読み取り終了

# 字句読み取り関数gettoken()

```
enum {
  SYM_NONE,
  SYM_EOF,
  SYM_ADD,
  …
  SYM_NUM,
  SYM_NAME
};

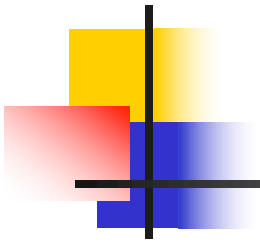
int  yynum;
char  yyname[100];

int
gettoken()
{
   int ch, ret_sym;

   ret_sym = SYM_NONE;
   while ((ch = fgetc(stdin))
           != EOF && isspace(ch))
         ;
   if (ch == EOF) {
         return  SYM_EOF;
   }
```

```
  if (isdigit(ch)) {
    ret_sym = SYM_NUM;
    yynum = 0;
    do {
      yynum *= 10;
      yynum += ch - '0';
    } while ((ch = fgetc(stdin)) != EOF
             && isdigit(ch));
    ungetc(ch, stdin);
  } else if (isalpha(ch)) {
    /* 識別子の読み取り。yynameに識別子を保存 */
  } else {
    yyname[0] = ch; yyname[1] = 0;
    switch (ch) {
      case  '+':
        ret_sym = SYM_ADD;
        break;
        …
      default:
        ret_sym = SYM_NONE;
      }
   }

   return  ret_sym;
}
```



# 字句解析器の自動生成

- 状態遷移機械なので状態遷移表さえ作ることができれば自動生成は可能
- どのようにして字句のルールから状態遷移表を生成するか？
  - 字句の表現方法（ルール）を定義する
    - ある「ルール」に則った文字列の「集合」として字句を定義する
    - 整数定数、浮動小数点定数、識別子など
  - そのルールに則っているかどうかをチェックする状態遷移機械（有限オートマトン）を生成する

# 基本的な用語の定義

- アルファベット (alphabet)
  - 記号 (symbol)の有限集合
    - {a, b, c, d}など
- 記号列
  - 記号の列（a, c, ba, abc, bbcddなど）
  - 一つも記号がない記号列をεとする
- 集合の積
  - AB = {xy| x∈A, y∈B} (A, Bは記号列の集合)
  - 例：A={a,b,cd}, B={ef, g}→ AB={aef,ag,bef,bg,cdef,cdg}
- 集合のべき

$$A^1 = A, A^2 = AA, A^3 = AAA = AA^2, A^n = AA^{n-1}$$

- 閉包 (closure)

$$A^+ = A^1 \bigcup A^2 \cdots \bigcup A^n \cdots = \bigcup_{i=1}^{\infty} A^i$$

1個以上の繰り返し

$$A^* = A^0 \bigcup A^1 \cdots \bigcup A^n \cdots = \bigcup_{i=0} A^i$$

0個以上の繰り返し

ある記号集合中の要素がどのように並べられ、新たな記号列集合を構成しうるか、という定義

各集合から要素を選んで並べていくときの組み合わせ全てを書き出す

A={a,b}なら
A*={ε,a,b,aa,bb,ab,aaa,aab,...}

# 正規表現 (regular expression)

- 字句の定義方法
  - Python, Ruby等、多くの使用例があり
- アルファベットAの正規表現とは
  - εは正規表現
  - Aの要素aは正規表現
  - RとSが正規表現ならR|S, RS, R*も正規表現
    - 「｜」は「または (or)」の意味
- アルファベットA上の正規表現Rの値L(R)
  - L(R)はRにより得られる文字列の集合
  - L(ε)={ε}
  - L(a)={a} (aはAの要素)
  - RとSが正規表現の場合
    L(R|S)=L(R)∪L(S), L(RS)=L(R)L(S), L(R*)=L(R)*

任意の正規表現が
3つの要素に分解できる

# 正規表現により表現される文字列集合の例



- L(ab)={ab}
- L(ccb|ab)={ccb,ab}
- L((ab)*)=L(ab)*={ε,ab,abab,ababab,…}
- L(c(cb|a)*b)={c}{cb,a}*{b}={cb,ccbb,cab,cacbb,caab,…}

# 演習問題1

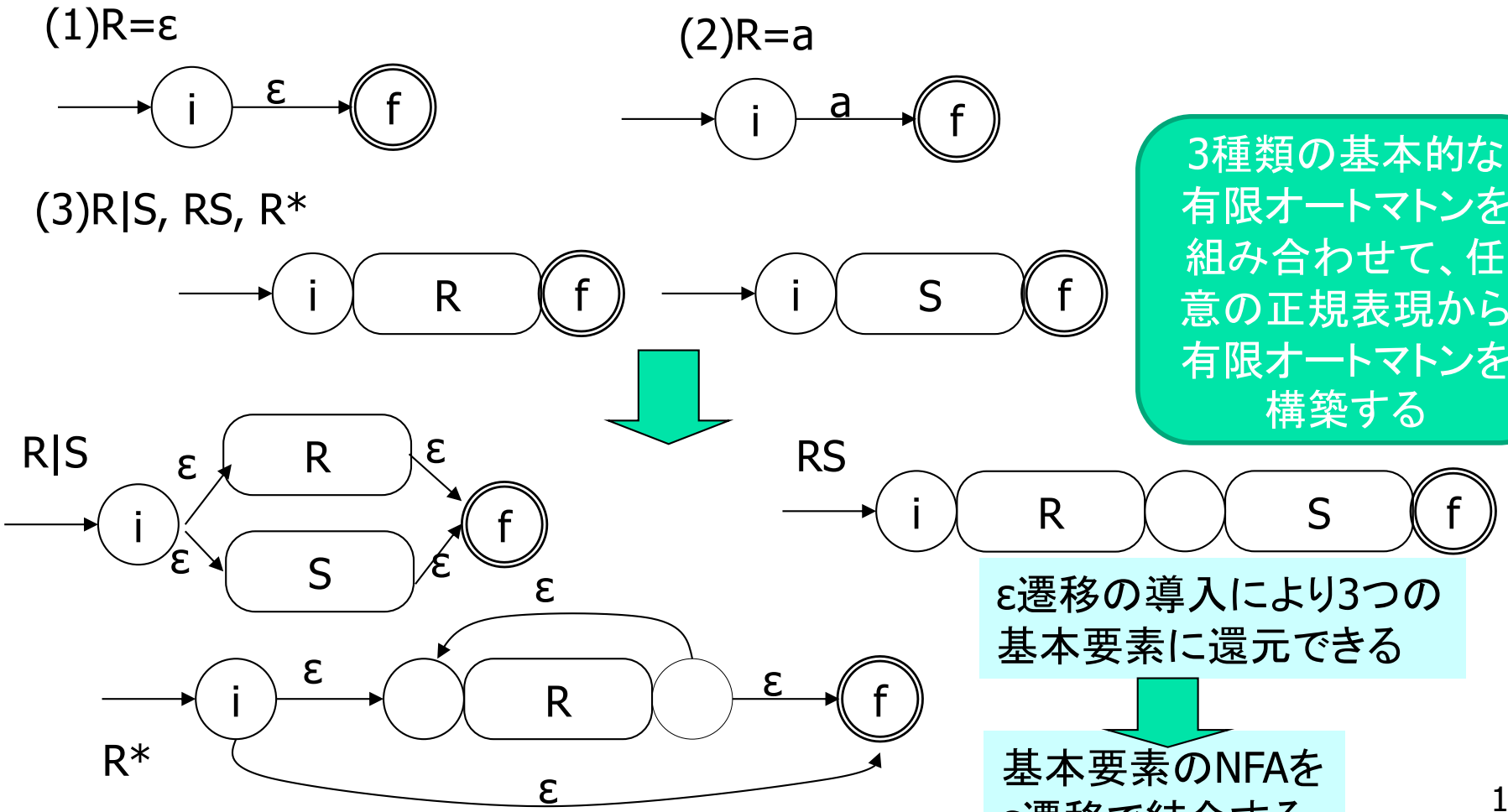
- <名前> → [a-zA-Z]([a-zA-Z]|[0-9])*に関して以下の問に答えよ
  - <名前>にマッチする文字列を以下から選べ
    - Compiler1
    - compiler@2　@が正規表現に含まれていないので不可
    - 2026cs　先頭が[a-zA-Z]でない数字なため不可
  - ただし、[a-z]はaからzまでの英語アルファベット、[A-Z]はAからZまでのアルファベット、[a-zA-Z]は大文字小文字全てのアルファベット（<英字>と表記する）、[0-9]は0から9までの数字（<数字>と表記する）を意味する

# 有限オートマトン (finite automaton)

- 与えられた記号列を読みながら、有限個の状態間を遷移しつつ、記号列を認識するもの
  - 入力：記号列、出力：マッチ（受理）したかどうか
- 記号列認識プログラムの自動化
  - 正規表現
    →非決定性有限オートマトン
    →決定性有限オートマトン
  - 非決定性と決定性
    - ε遷移が有る（非決定性）か無い（決定性）か　遷移先が決定できない場合を許容する
    - 複数のオートマトンを接続しやすい（非決定性）か
      接続しにくい（決定性）か

# 正規表現から非決定性有限オートマトン (NFA)

- NFA: nondeterministic finite automaton



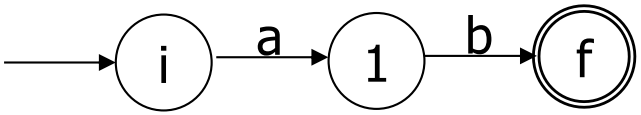

3種類の基本的な有限オートマトンを組み合わせて、任意の正規表現から有限オートマトンを構築する

ε遷移の導入により3つの基本要素に還元できる

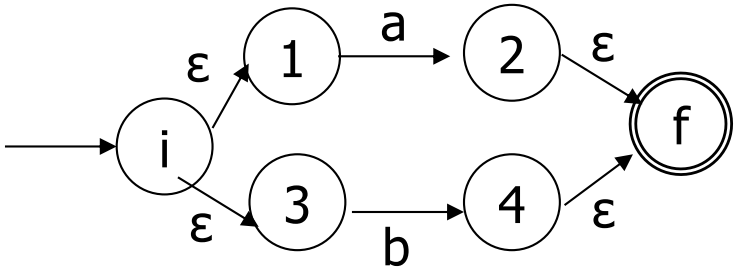
基本要素のNFAをε遷移で結合する

# 正規表現からNFAを作る例(1)

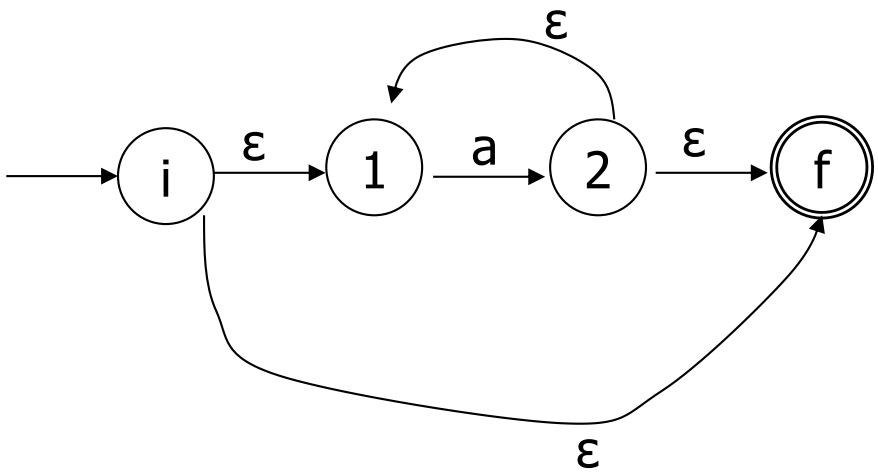
- ab




- a|b

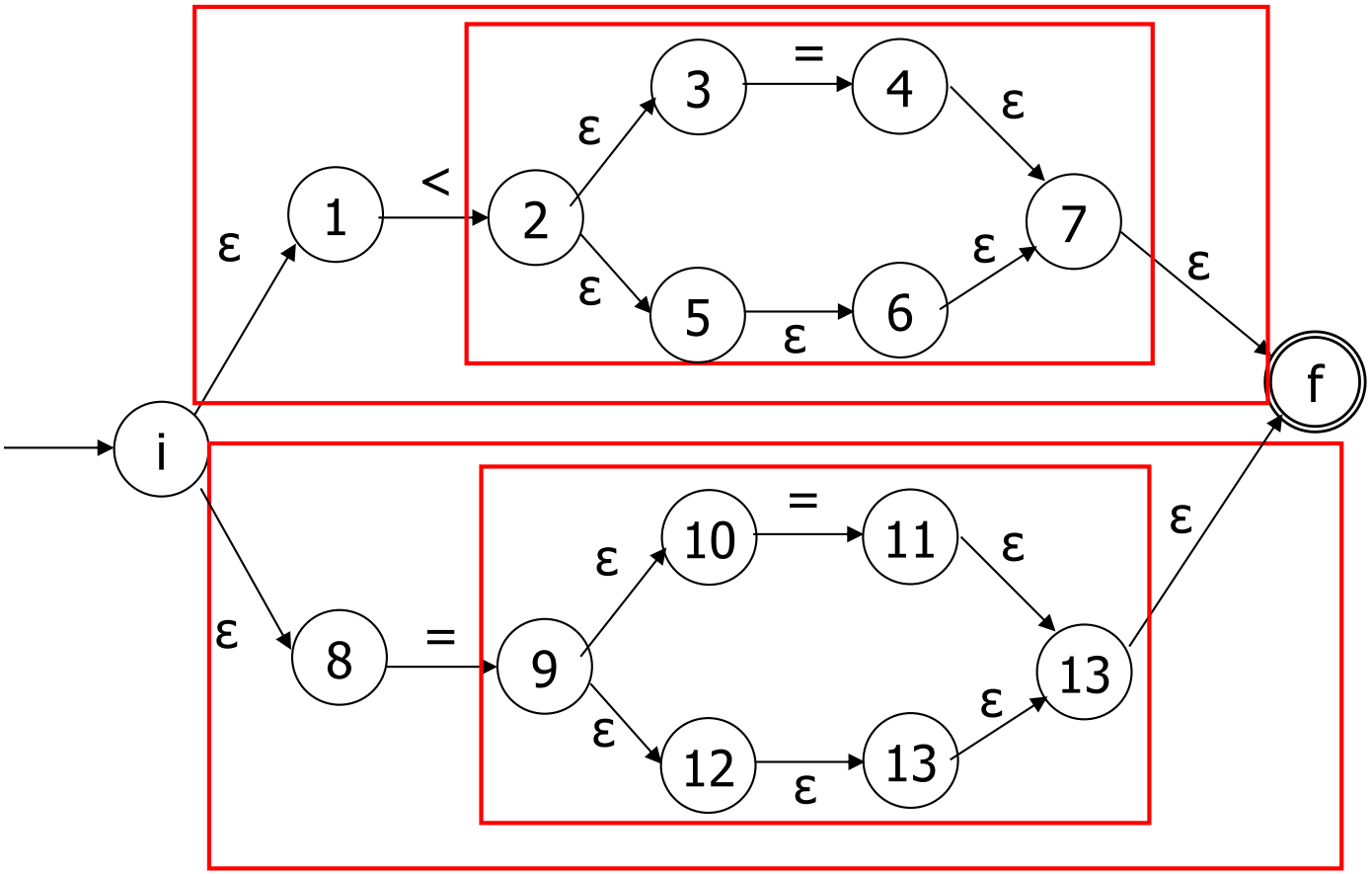



- a*



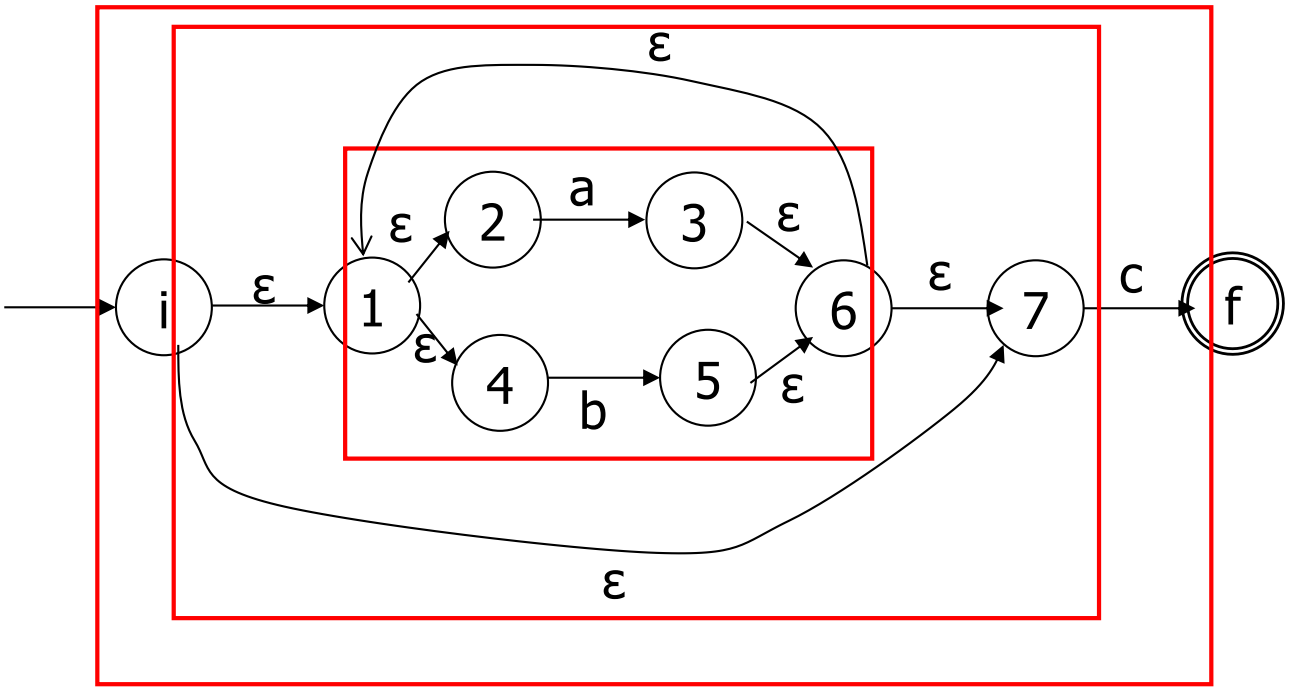
# 正規表現からNFAを作る例(2)

- (<(=|ε))|(=(=|ε))



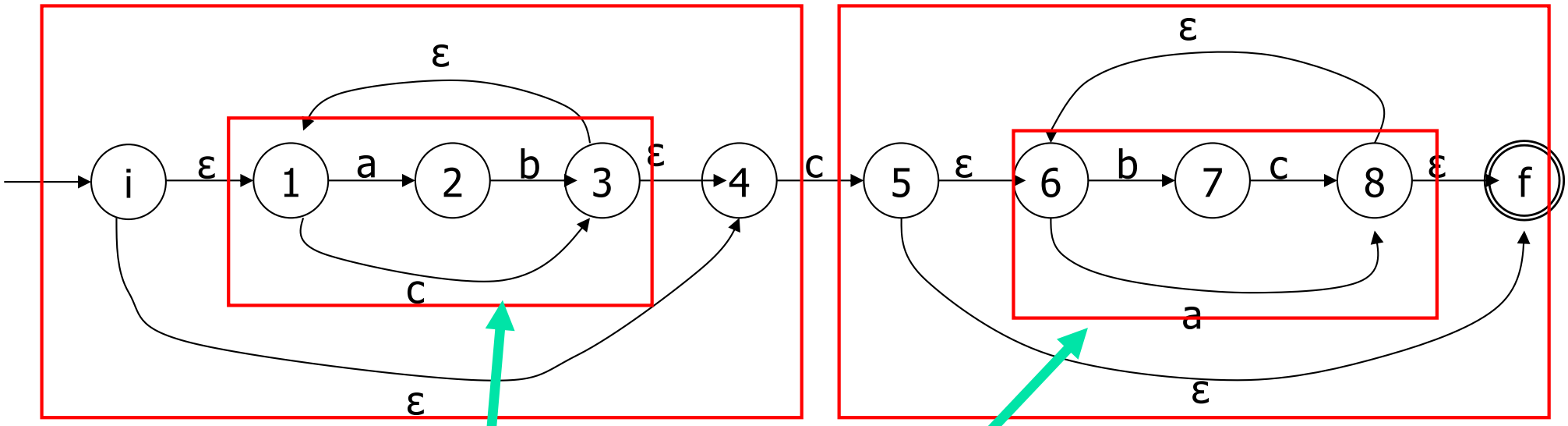
# 正規表現からNFAを作る例(3)

- (a|b)*c



# 正規表現からNFAを作る例(4)

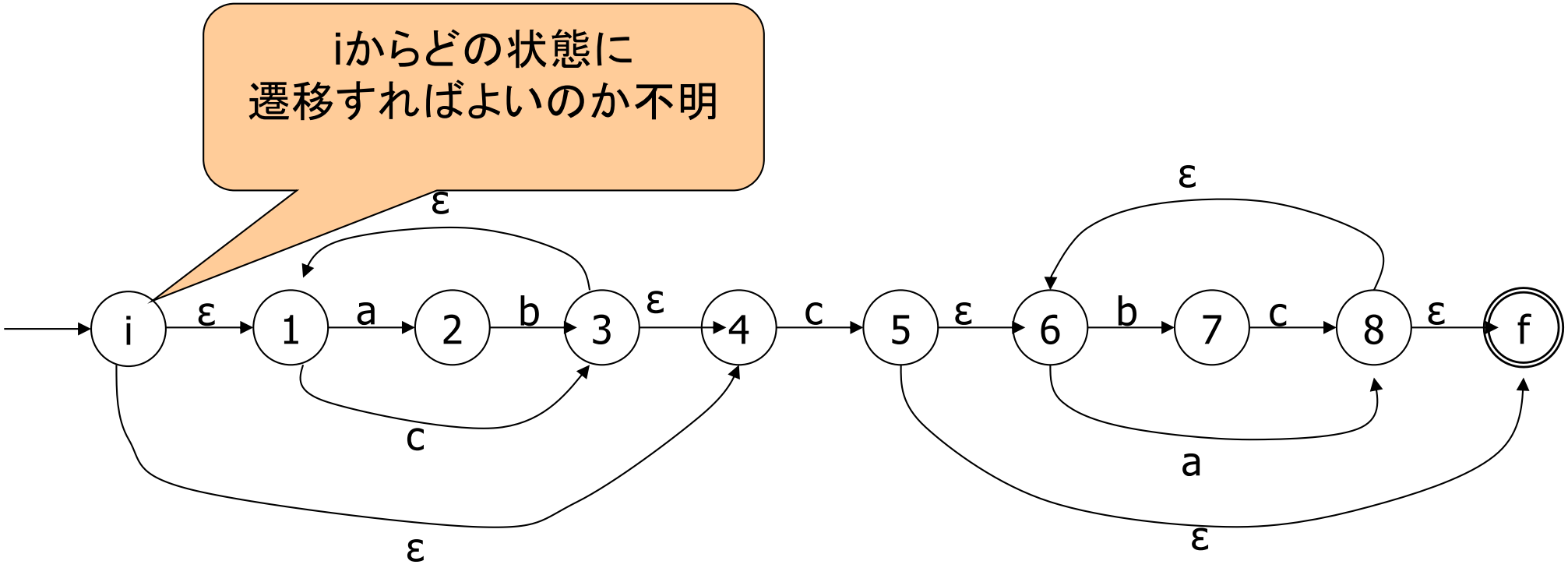
- (ab|c)*c(bc|a)*



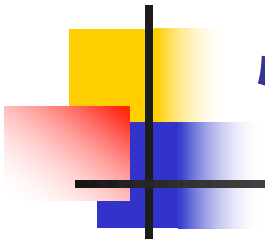

ab及びcに至るεの状態遷移は
図の簡略化のため削除した
（後述のDFAへの変換を参照）

こちらも同様

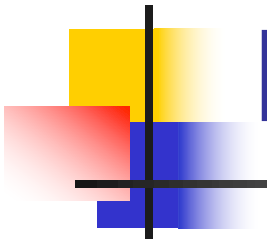
# NFAにおける非決定性



# 非決定性有限オートマトンから決定性有限オートマトンへ (DFA)



- deterministic finite automaton
  - εによる遷移がない
  - 一つの状態から同じ記号による異なった状態への遷移がない
- 先の例
  - iから1へも4へも遷移できる（εで接続されている）
  - i,1,4を一つの状態として定義する
  - 2からbで到達する3も1,4と共に一つの状態として定義する



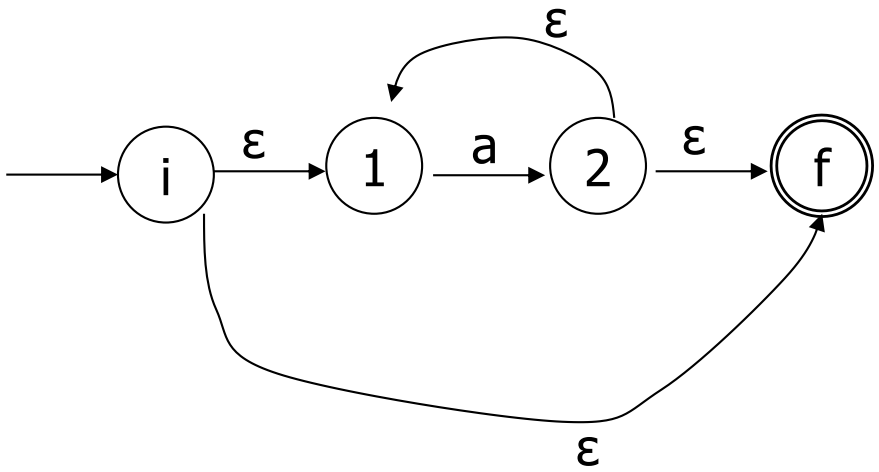
# DFAの作成法

- (1) NFAの初期状態とその初期状態からε遷移で辿れる全ての状態からなる集合をDFAの初期状態とする
- (2) 状態の集合からの遷移先を、その遷移先からε遷移で辿れる遷移先まで含めて全て合併する。同一の記号で遷移できる状態も合併する
- (3) (2)を新しい集合および遷移が得られなくなるまで繰り返す

状態数の最小化も考えてみよう

# NFAからDFAの例(1)

- a*



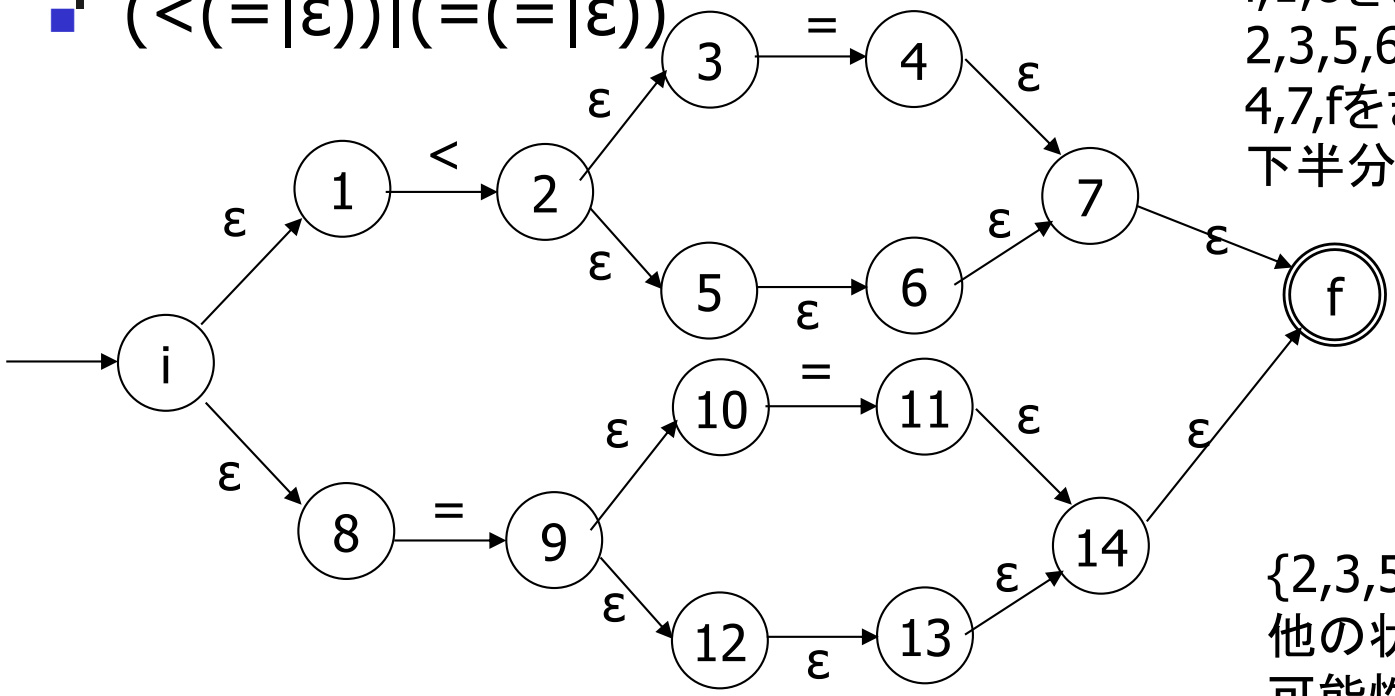

εで遷移で結ばれるi, 1, fを一つの状態、
2, 1, fをもう一つの状態に纏める
2,1,fの状態では1→2の遷移を
自分自身への遷移として表現する

aで遷移する状態を一つに纏める

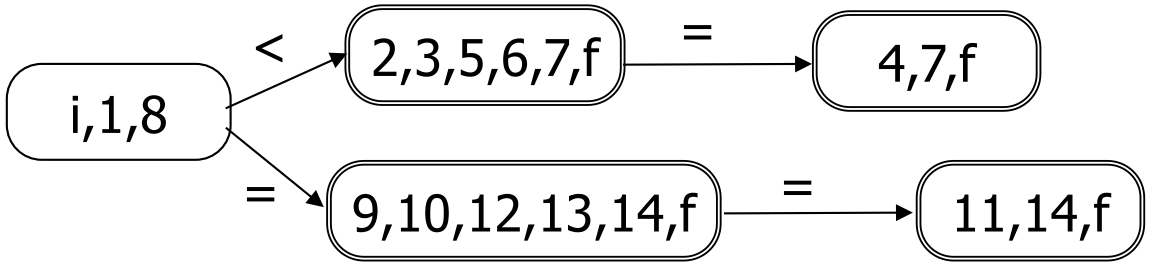


# NFAからDFAの例(2)

- (<(=|ε))|(=(=|ε))

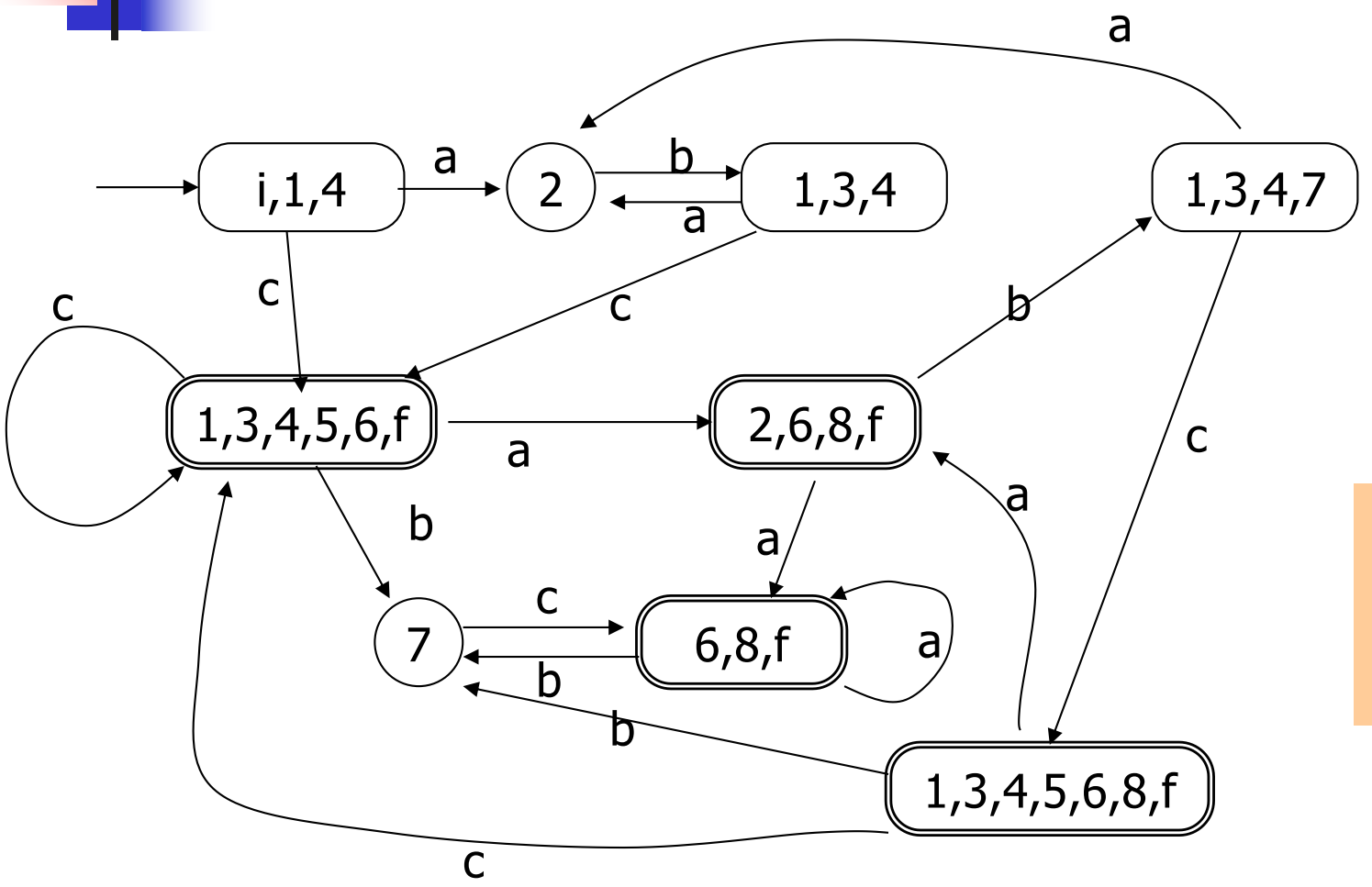



i,1,8をまとめる
2,3,5,6,7,fをまとめる
4,7,fをまとめる
下半分も同様




{2,3,5,6,7,f}のように、他の状態に遷移する可能性のある受理状態は、次の状態に遷移するための入力以外の入力（例えば=以外）により受理となる。

# 先の例は？




字句解析器の
自動生成が
可能となる

# 先のDFA作成例を途中まで

- 初期状態
  - iからε遷移する1,4も含めた(i,1,4)
- (i,1,4)からの遷移先
  - a遷移: 1→2
  - c遷移: 1→3, 4→5 ⇒ 3, 5もまとめる
    - 3からε遷移する1,4及び
      5からε遷移する6,fもまとめる⇒(1,3,4,5,6,f) (a*)
- 2からの遷移先
  - b遷移: 2→3
    - 3からのε遷移1,4をまとめる⇒(1,3,4)
- (1,3,4)からの遷移先
  - a遷移: 1→2
  - c遷移: (a*)と同じ
- 以下、繰り返し

# lex (flex)のサンプル

```
%{
#include <stdio.h>
enum {
SYM_NONE, SYM_ADD, SYM_SUB, SYM_MUL,
SYM_DIV, SYM_CR, SYM_INTEGER
};

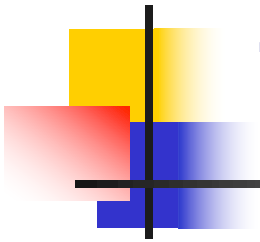
char token_kind[][5] = {"none", "add",
"sub", "mul", "div", "cr", "int"};
int yylval;
%}

%%
"+"     return  SYM_ADD;
"-"     return  SYM_SUB;
"*"     return  SYM_MUL;
"/"     return  SYM_DIV;
"¥n"    return  SYM_CR;
[0-9]+  {
          yylval = atoi(yytext);
          return  SYM_INTEGER;
        }
%%
```

```
/* flex tinycalc.l
   gcc -o sample01 lex.yy.c -lfl */
int
main(int arc, char** argv)
{
  int token;

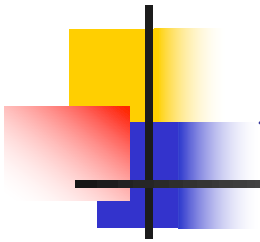
  yyin = stdin;
  while ((token = yylex())!=0) {
    printf("token(%s): val(%d),
text(%s)¥n”,
            token_kind[token],
            yylval, yytext);
  }
}
```

“%%”と”%%”の間に
**正規表現**と**その正規表現に**
**マッチした場合の処理(Cのコード)**
を並べる



# まとめ

- 字句解析の概要
  - 字句解析のプログラム例
  - DFA,NFA



# 演習問題2

- 以下の正規表現に対応するNFAを作成せよ。次にDFAを作成せよ。ただし、digitは0-9の数字一文字，”.”は小数点である。（digitを一つのアルファベットとして扱ってかまわない）
  - digit digit*
  - digit digit* (. digit digit* | ε)